Introduction to Cybersecurity

Software

Jasmin Azemović
CISO | Associate Professor | Microsoft MVP

Why Software?

- Why is software as important to security as crypto, access control, protocols?
- Virtually all of information security is implemented in software
- If your software is subject to attack, your security can be broken
  - Regardless of strength of crypto, access control or protocols
- Software is a poor foundation for security

Bad Software is Ubiquitous

- NASA Mars Lander (cost $165 million)
  - Crashed into Mars due to…
  - …error in converting English and metric units of measure
  - Believe it or not
- Denver airport
  - Baggage handling system --- very buggy software
  - Delayed airport opening by 11 months
  - Cost of delay exceeded $1 million/day
  - What happened to person responsible for this fiasco?

Complexity

- “Complexity is the enemy of security”, Paul Kocher, Cryptography Research, Inc.

System	Lines of Code (LOC)
Netscape	17 million
Space Shuttle	10 million
Linux kernel 2.6.0	5 million
Windows XP	40 million
Mac OS X 10.4	86 million
Boeing 777	7 million

- A new car contains more LOC than was required to land the Apollo astronauts on the moon

Lines of Code and Bugs

- Conservative estimate: 5 bugs/10,000 LOC
- **Do the math**
  - Typical computer: 3k exe's of 100k LOC each
  - Conservative estimate: 50 bugs/exe
  - So, about 150 000 bugs per computer
  - So, 30,000-node network has 4.5 billion bugs
  - Maybe only 10% of bugs security-critical and only 10% of those remotely exploitable
  - Then "only" 45 million critical security flaws!

Software Security Topics

- Program flaws (unintentional)
  - Buffer overflow
  - Incomplete mediation
- Malicious software (intentional)
  - Viruses
  - Worms
  - Other breeds of malware

Buffer Overflow



Possible Attack Scenario

- Users enter data into a Web form
- Web form is sent to server
- Server writes data to array called buffer, without checking length of input data
- Data "overflows" buffer
  - Such overflow might enable an attack
  - If so, attack could be carried out by anyone with Internet access

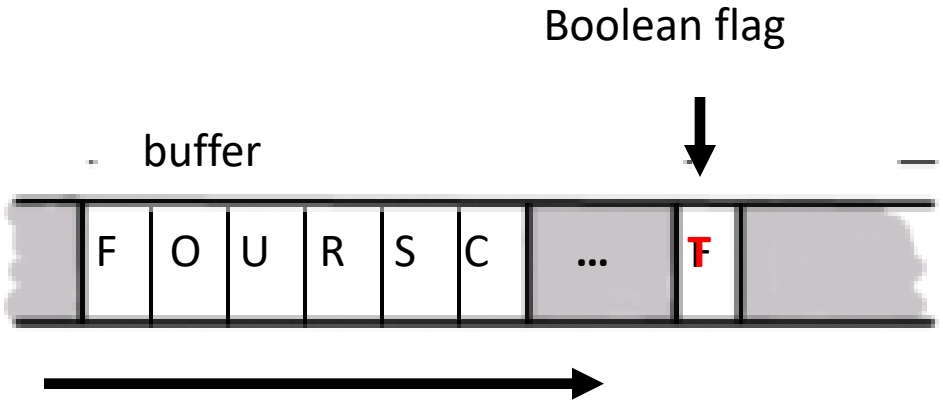
Buffer Overflow

```
int main(){
    int buffer[10];
    buffer[20] = 37;}
```

- **Q:** What happens when code is executed?
- **A:** Depending on what resides in memory at location "buffer[20]"
  - Might overwrite **user** data or code
  - Might overwrite **system** data or code
  - Or program could work just fine

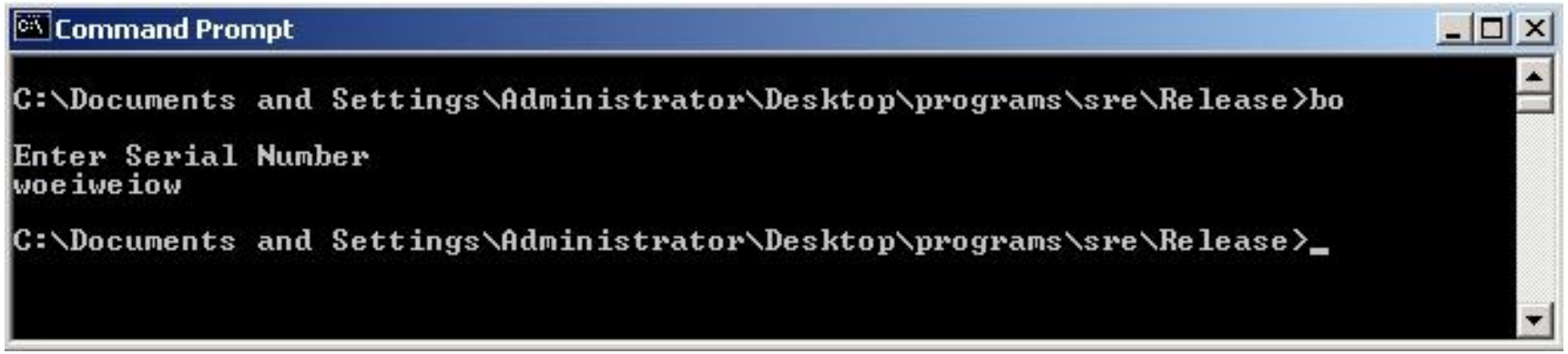
Simple Buffer Overflow

- Consider boolean flag for authentication
- Buffer overflow could overwrite flag allowing anyone to authenticate



Stack Smashing Example

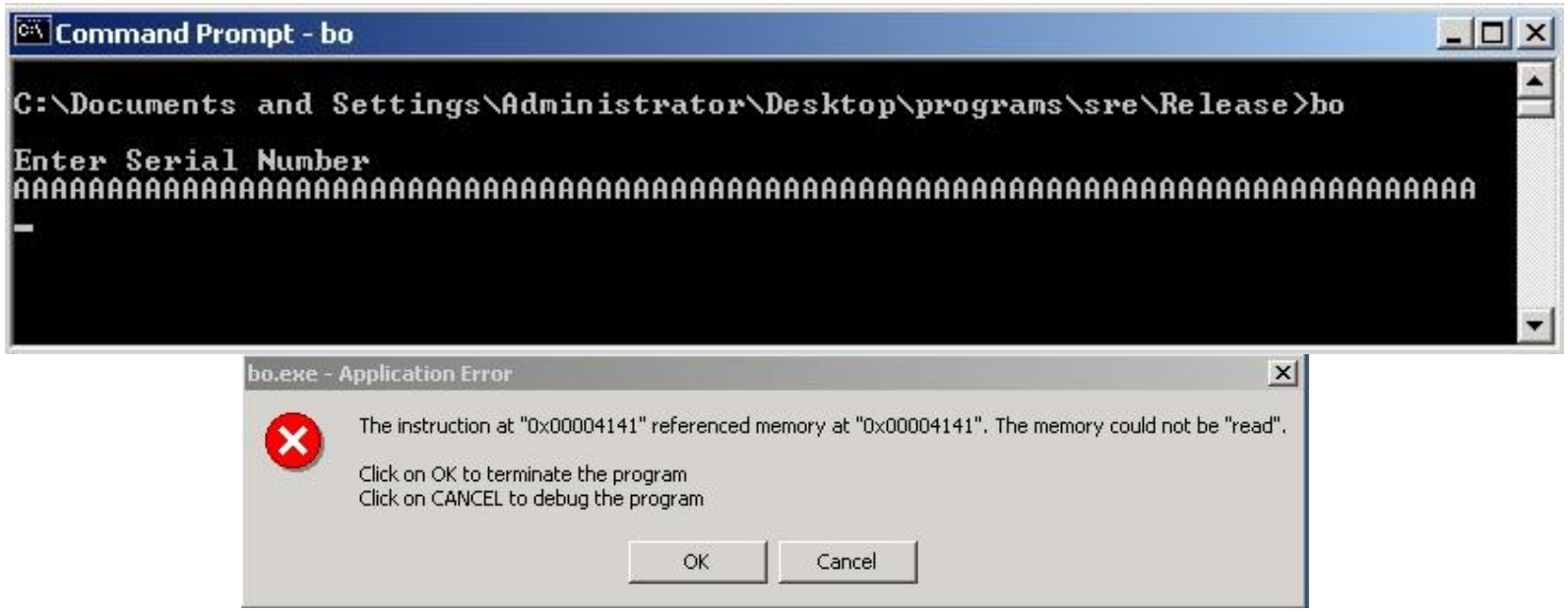
- Program asks for a serial number that the attacker does not know
- Attacker does **not** have source code
- Attacker does have the executable (exe)



- Program quits on incorrect serial number

Buffer Overflow Present?

- By trial and error, attacker discovers apparent buffer overflow



- Note that 0x41 is ASCII for “A”

Disassemble Code

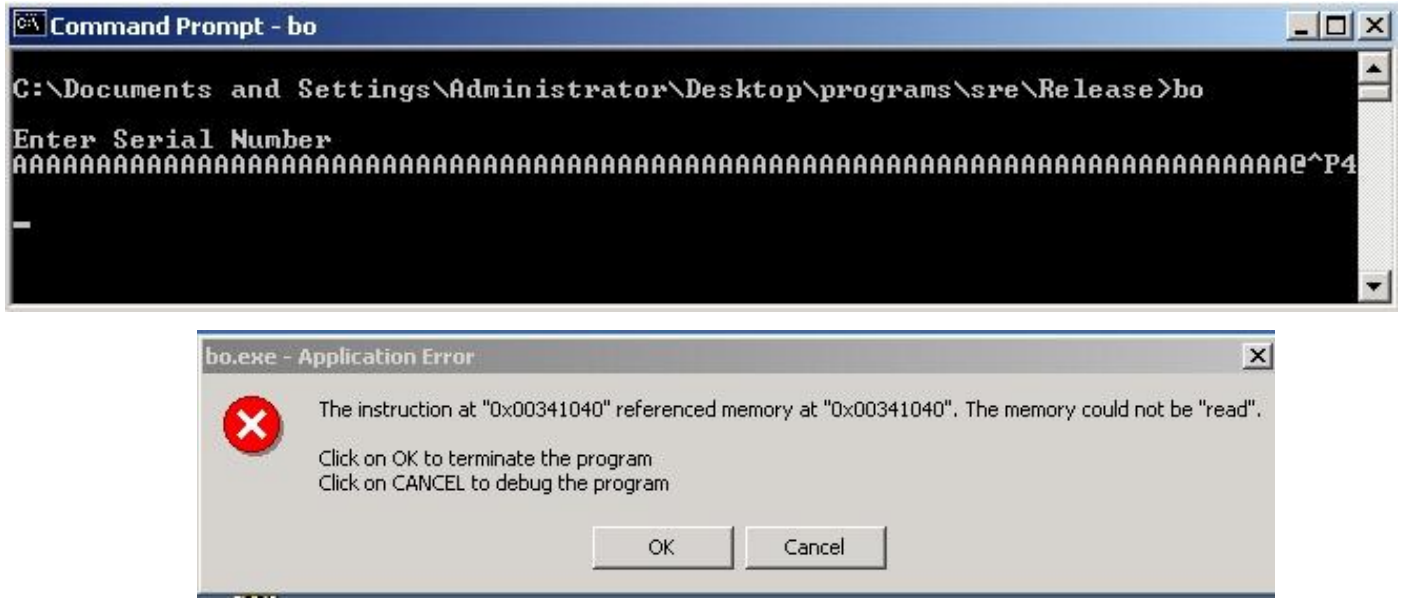
- Next, disassemble bo.exe to find

```
.text:00401000
.text:00401000                sub     esp, 1Ch
.text:00401003                push    offset aEnterSerialNum ; "\nEnter Serial Number\n"
.text:00401008                call    sub_40109F
.text:0040100D                lea     eax, [esp+20h+var_1C]
.text:00401011                push    eax
.text:00401012                push    offset aS       ; "%s"
.text:00401017                call    sub_401088
.text:0040101C                push    8
.text:0040101E                lea     ecx, [esp+2Ch+var_1C]
.text:00401022                push    offset aS123n456 ; "S123N456"
.text:00401027                push    ecx
.text:00401028                call    sub_401050
.text:0040102D                add     esp, 18h
.text:00401030                test    eax, eax
.text:00401032                jnz     short loc_401041
.text:00401034                push    offset aSerialNumberIs ; "Serial number is correct.\n"
.text:00401039                call    sub_40109F
.text:0040103E                add     esp, 4
```

- The goal is to exploit buffer overflow to jump to address 0x401034

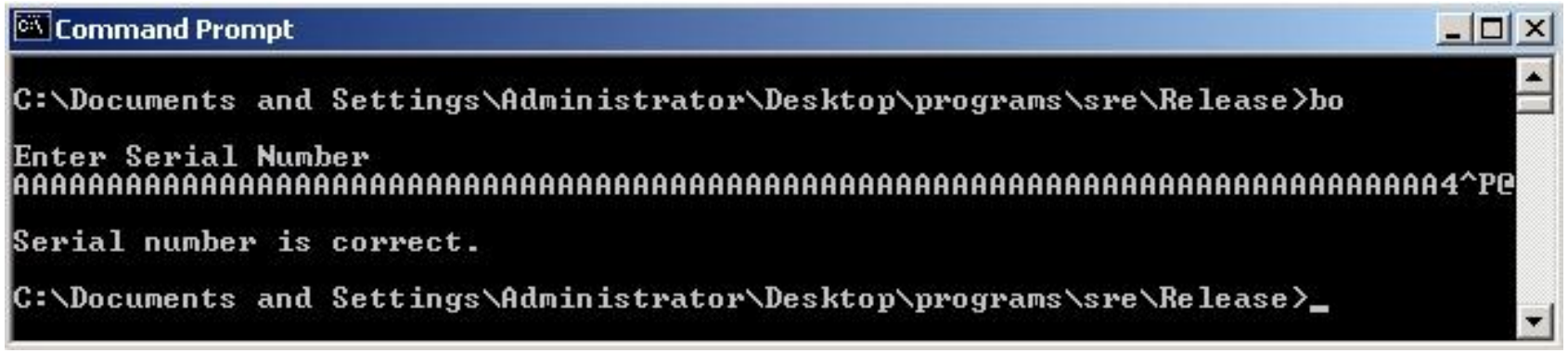
Buffer Overflow Attack

- Find that, in ASCII, 0x401034 is “@^P4”


Command Prompt - bo
C:\Documents and Settings\Administrator\Desktop\programs\sre\Release>bo
Enter Serial Number
AAAA

Overflow Attack, Take 2

- Reverse the byte order to “4^P@” and…



- Success! We’ve bypassed serial number check by exploiting a buffer overflow
- What just happened?
  - Overwrote return address on the stack

Buffer Overflow

- Attacker did **not** require access to the source code
- Only tool used was a disassembler to determine address to jump to
- Find desired address by trial and error?
  - Necessary if attacker does not have exe
  - For example, a remote attack

Source Code

- Source code for buffer overflow example

```
#include <stdio.h>
#include <string.h>

main()
{
    char in[75];

    printf("\nEnter Serial Number\n");

    scanf("%s", in);

    if(!strncmp(in, "S123N456", 8))
    {
       printf("Serial number is correct.\n");
    }
}
```

Input Validation

- Consider web form data
- Suppose input is validated on client
- For example, the following is valid

```
http://www.things.com/orders/final&custID=112&num=55A&qt
  y=20&price=10&shipping=5&total=205
```

- Suppose input is not checked on server
  - Why bother since input checked on client?
  - Then attacker could send http message

```
http://www.things.com/orders/final&custID=112&num=55A&qt
  y=20&price=10&shipping=5&total=25
```

Malware

Malicious Software

- Malware is not new…
- Types of malware (lots of overlap)
  - **Virus** — passive propagation
  - **Worm** — active propagation
  - Trojan horse — unexpected functionality
  - Trapdoor/backdoor — unauthorized access
  - Rabbit — exhaust system resources

Where do Viruses Live?

- They live just about anywhere, such as…
- Boot sector
  - Take control before anything else
- Memory resident
  - Stays in memory
- Applications, macros, data, etc.
- Library routines
- Compilers, debuggers, virus checker, etc.
  - These would be particularly nasty!

Malware Examples

- Brain virus (1986)
- Morris worm (1988)
- Code Red (2001)
- SQL Slammer (2004)
- Botnets (currently fashionable)
- Future of malware?

Brain

- First appeared in 1986
- More annoying than harmful
- A prototype for later viruses
- Not much reaction by users
- What it did
  1. Placed itself in boot sector (and other places)
  2. Screened disk calls to avoid detection
  3. Each disk read, checked boot sector to see if boot sector infected; if not, goto 1
- Brain did nothing really malicious

Morris Worm

- First appeared in 1988
- What it tried to do
  - Determine where it could spread, then…
  - …spread its infection and…
  - …remain undiscovered
- Morris claimed his worm had a „bug“!
  - It tried to re-infect infected systems
  - Led to resource exhaustion
  - Effect was like a so-called rabbit

Code Red Worm

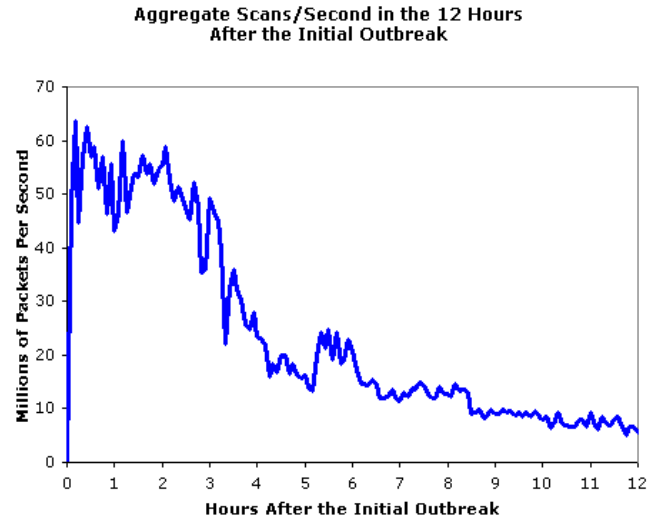
- Appeared in July 2001
- Infected more than **250,000 systems in about 15 hours**
- Eventually infected 750,000 out of about 6,000,000 vulnerable systems
- Exploited buffer overflow in Microsoft IIS server software
  - Then monitor traffic on port 80, looking for other susceptible servers

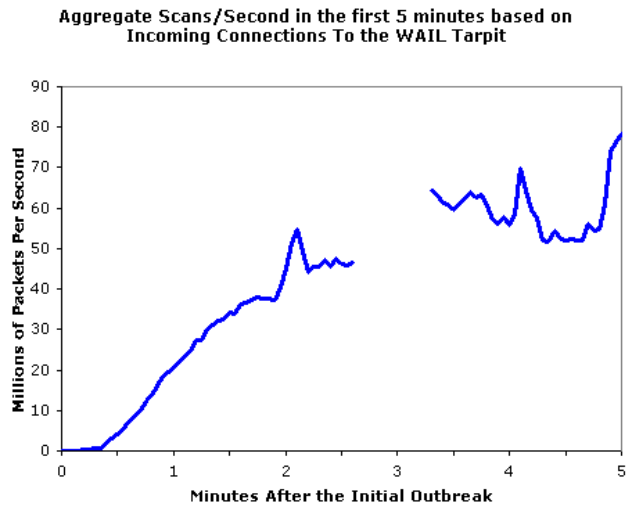
Code Red: What it Did

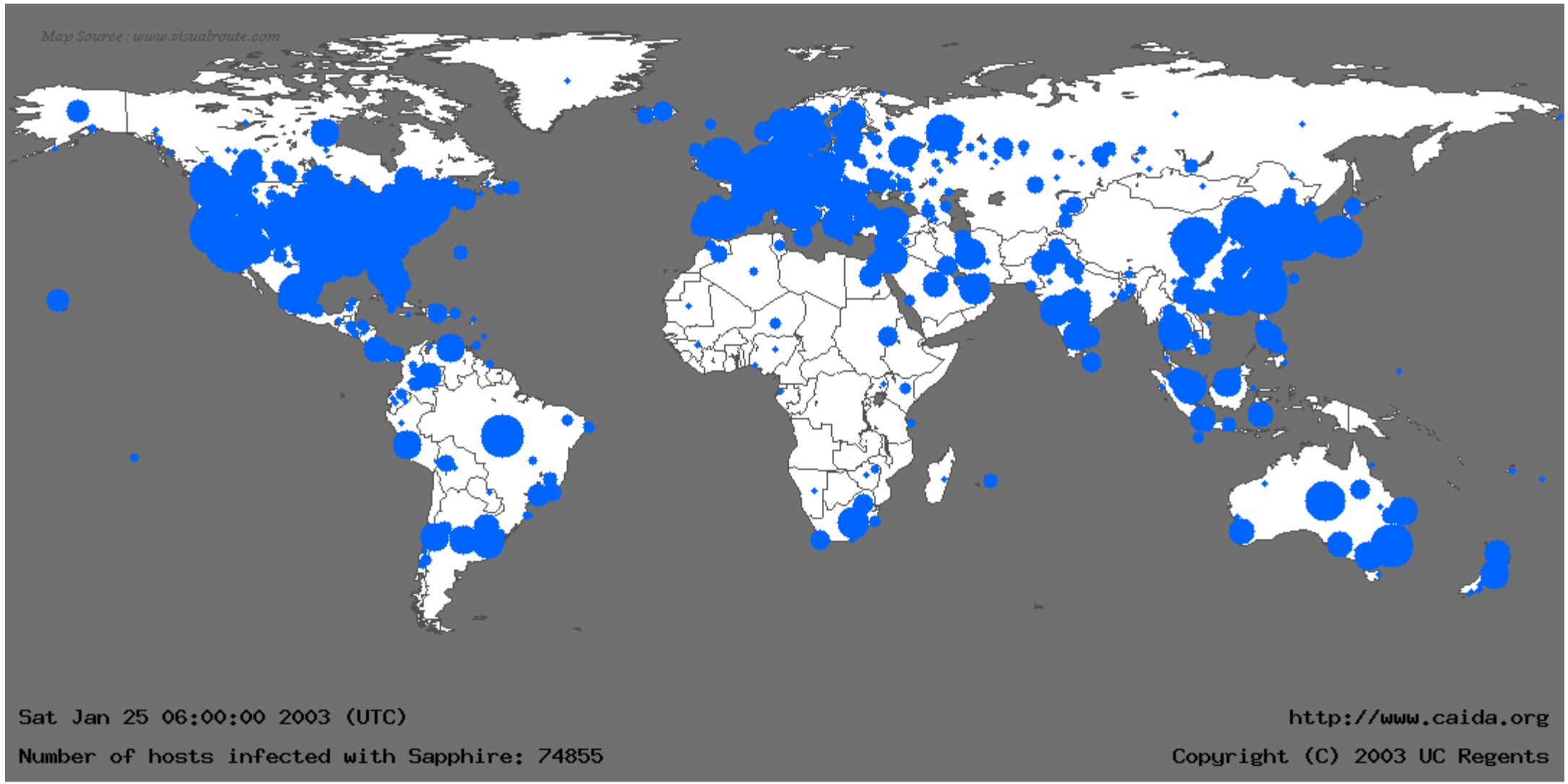
- Day 1 to 19 of month: spread its infection
- Day 20 to 27: distributed denial of service attack (DDoS) on `www.whitehouse.gov`
- Later version (several variants)
  - Included trapdoor for remote access
  - Rebooted to flush worm, leaving only trapdoor
- Some say it was “beta test for info warfare”
  - But no evidence to support this

SQL Slammer

- Infected **75,000 systems in 10 minutes!**
- At its peak, infections doubled every 8.5 seconds
- Spread “too fast”…
- …so it “burned out” available bandwidth






Map Source : www.visualroute.com
Sat Jan 25 06:00:00 2003 (UTC)
Number of hosts infected with Sapphire: 74855
http://www.caida.org
Copyright (C) 2003 UC Regents

Why was Slammer Successful?

- Worm size: **one 376-byte UDP packet**
- Firewalls often let one packet thru
  - Then monitor ongoing “connections”
- Expectation was that much more data required for an attack
  - So no need to worry about 1 small packet
- Slammer defied “experts”

Malware Detection

- Three common detection methods
  - Signature detection
  - Change detection
  - Anomaly detection
- We briefly discuss each of these
  - And consider advantages…
  - …and disadvantages

Signature Detection

- A **signature** may be a string of bits in exe
  - Might also use wildcards, hash values, etc.
- For example, W32/Beast virus has signature

  83EB 0274 EB0E 740A 81EB 0301 0000
  - That is, this string of bits appears in virus
- We can search for this signature in all files
- If string found, have we found W32/Beast?
  - Not necessarily — string could appear elsewhere
  - At random, chance is only $1/2^{112}$
  - But software is not random

Signature Detection

- Advantages
  - Effective on “ordinary” malware
  - Minimal burden for users/administrators
- Disadvantages
  - Signature file can be large (10,000’s)…
  - …making scanning slow
  - Signature files must be kept up to date
  - ***Cannot detect unknown viruses***
  - Cannot detect some advanced types of malware
- The most popular detection method

Future of Malware

- Recent trends
  - Encrypted, polymorphic, metamorphic malware
  - Fast replication/Warhol worms
  - Flash worms, slow worms
  - Botnets
- The future is bright for malware
  - Good news for the bad guys…
  - …bad news for the good guys
- Future of malware detection?

